04

Part 4 · Caching & Memory

Reusing results and understanding where executor memory goes.

- cache() & persist()
- Storage levels
- Materialisation · unpersist
- Memory model
- Spill
- Executor sizing

4.1 Caching DataFrames

★★★

IN ONE LINE

cache() stores a computed DataFrame in memory so repeated actions skip the recomputation.

By default Spark **recomputes** a DataFrame from its source every time an action needs it. `cache()` keeps the computed partitions in executor memory so later actions reuse them instead.

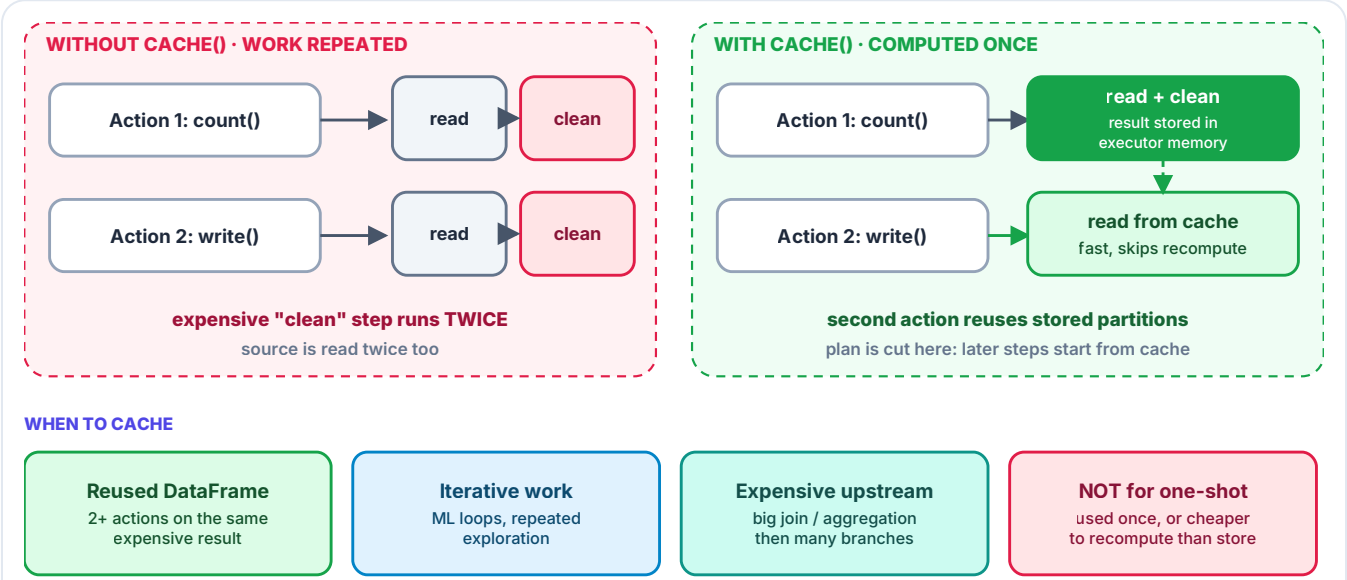


Figure 4.1 — Recompute every time vs compute once and reuse

TYPICAL PATTERN

```
features = (raw.join(dim, 'id')
               .filter('valid')
               .groupBy('user').agg(F.sum('amt').alias('total')))

features.cache()          # only marks it: nothing is stored yet
features.count()          # action: computes AND fills the cache

features.write.parquet('/out/a')                # reuses cache
features.filter('total > 100').show()           # reuses cache

features.unpersist()      # free memory when done
```

Cache when...	Do not cache when...
The same DataFrame feeds **several actions** or branches	It is used **once** (cache just costs memory)
The upstream work is **expensive** (big join, aggregation, UDF)	Recomputing is cheap, e.g. a simple Parquet scan with filter
It fits in memory (or you accept disk spill)	It is huge and would evict other useful data
You iterate on it (ML, exploratory analysis)	Data changes underneath and freshness matters

COMMON TRAP

Caching a DataFrame that is only used once **slows the job**: you pay to store it and never read it. Measure before and after (9.2). On Databricks, the **disk cache** (7.7) already speeds repeated reads of the same files without `cache()`.

4.2 Persistence and Storage Levels

★★☆

IN ONE LINE

persist() chooses where cached data lives: memory, disk, replicated or off-heap.

`cache()` is shorthand for `persist()` with the default storage level. `persist()` lets you choose **where and how** the data is kept: memory, disk, replicated, or off-heap.

Storage level	In memory	On disk	Replicas	Notes
MEMORY_ONLY	yes	no	1	evicted partitions are recomputed
MEMORY_AND_DISK	yes	overflow	1	DEFAULT for DataFrame.cache()
DISK_ONLY	no	yes	1	slow, but frees memory
MEMORY_AND_DISK_2	yes	overflow	2	copy on 2 nodes: fault tolerance
OFF_HEAP	off-heap	overflow	1	outside JVM heap, less GC

EVICTION

Cached blocks are evicted least-recently-used when memory is needed.
MEMORY_ONLY blocks are dropped (recomputed later); MEMORY_AND_DISK blocks move to disk.

Scala/Java also offer *_SER variants (serialised, more compact, more CPU). DataFrame cache uses a compact columnar format.

Figure 4.2 — Common storage levels

CHOOSING A LEVEL

```
from pyspark import StorageLevel

df.cache()                                  # MEMORY_AND_DISK for DataFrames
df.persist(StorageLevel.DISK_ONLY)          # large data, frees executor memory
df.persist(StorageLevel.MEMORY_AND_DISK_2)  # replicated for faster recovery
print(df.storageLevel)                      # check
```

Situation	Choose
Default: fits in memory or spilling is acceptable	`cache()` / MEMORY_AND_DISK
Data much bigger than memory, reused several times	`DISK_ONLY` (or just write an intermediate Delta table)
Long job where recomputing after a failure is costly	`*_2` replicated levels
Very long lineage or need to truncate the plan	**checkpoint()** (writes to reliable storage, cuts lineage)

INTERVIEW ANGLE

cache vs checkpoint: cache keeps data on executors for speed and keeps lineage; checkpoint writes to storage and *cuts* lineage (useful for iterative algorithms with huge plans). For large reusable intermediates, **writing a Delta table** is often better than any cache.

4.3 Cache Materialization and unpersist()

★★☆

IN ONE LINE

cache() is lazy: the first full action fills it; unpersist() releases it.

Because Spark is lazy, `cache()` does **not** run anything: it only marks the DataFrame. The cache is filled (**materialised**) the first time an action computes it. Release it with `unpersist()` when you are done.

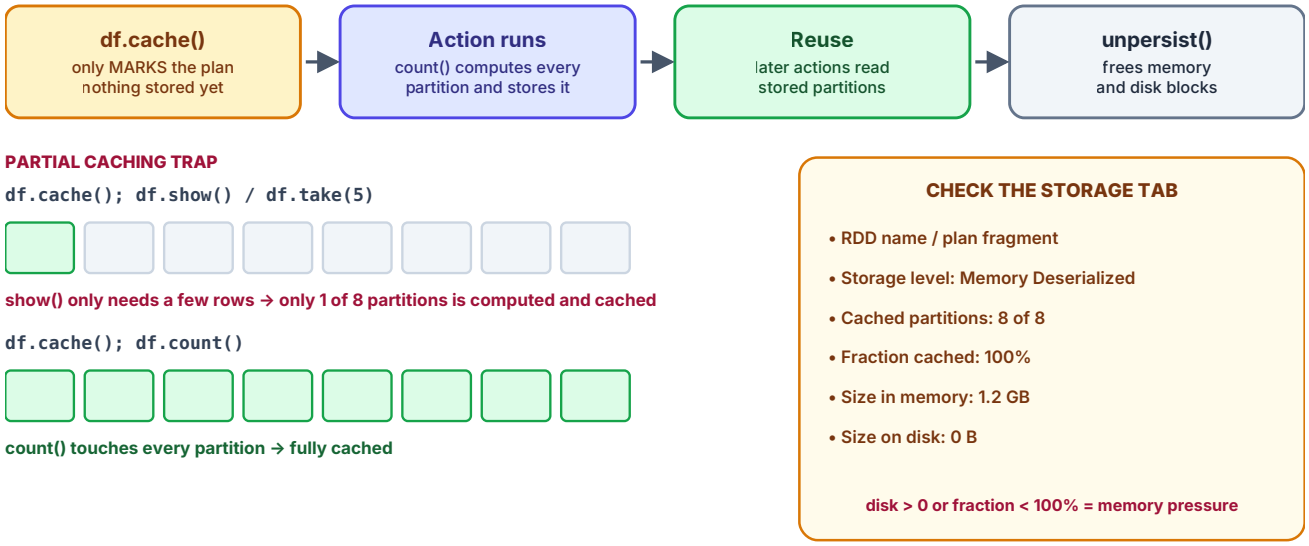


Figure 4.3 — Mark, materialise, reuse, release

Materialise fully

Trigger with an action that touches **every partition**: `df.count()` or a no-op write. `show()`, `take()` and `limit()` compute only the partitions they need.

Release explicitly

`df.unpersist()` frees executor memory immediately (`blocking=True` waits). Spark also evicts LRU blocks automatically, but explicit release prevents memory pressure and spill for the following stages.

FULL LIFECYCLE

```
df = expensive_transform(raw).cache()
df.count()                # materialise: every partition computed once

out1 = df.groupBy('a').count()
out2 = df.groupBy('b').count()
out1.write.parquet('/o1'); out2.write.parquet('/o2')

df.unpersist()            # done: give memory back
```

COMMON TRAP

Forgetting `unpersist()` in a long notebook or job keeps old data pinned, leaving less room for execution memory later. And if the underlying source changes after caching, you may read **stale** data.

4.4 Memory Management and Garbage Collection ★★★

IN ONE LINE

Executor memory splits into execution, storage, user and overhead regions; pressure shows up as spill, GC and OOM.

Each executor's memory is split into regions. Understanding it explains OOM errors, spill and GC trouble.

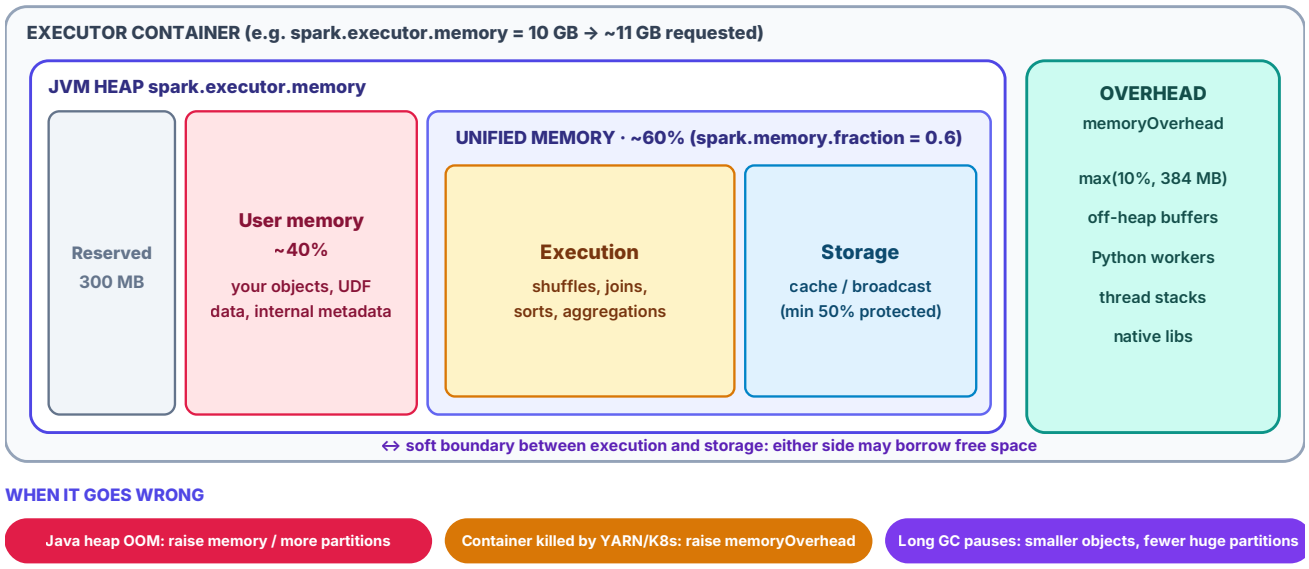


Figure 4.4 — Layout of executor memory (unified memory model)

Region	Used for	Key setting
Execution	Shuffle buffers, hash tables for joins and aggregations, sorting	Part of unified memory; can evict cache if needed
Storage	Cached DataFrames, broadcast variables	`spark.memory.storageFraction` = 0.5 (protected share)
User memory	Your own data structures, UDF objects	Remainder of heap (≈ 40%)
Overhead	Non-JVM memory: Python workers, native buffers	`spark.executor.memoryOverhead` (≥ 10% / 384 MB)

Garbage collection (GC)

The JVM pauses work to free memory. In the Spark UI **GC time > ~10%** of task time signals pressure. Causes: huge partitions, big collections in user code, caching too much, too many objects. Prefer DataFrame API (compact Tungsten format).

Typical memory errors

- **java.lang.OutOfMemoryError: Java heap space** — task needs more than it has.
- **Container killed … exceeding memory limits** — raise `memoryOverhead` (often PySpark).
- **Driver OOM** — `collect()`, huge broadcast.

INTERVIEW ANGLE
Remember the numbers: **300 MB reserved, 60% unified, 50% of that protected for storage, overhead max(10%, 384 MB)**. And the fix order: more partitions → fix skew → bigger executors.

4.5 Spill to Disk

★★★

IN ONE LINE
When a task cannot fit its working data in memory it spills to disk: slower, but safer than OOM.

Spill happens when an operation (sort, shuffle, join, aggregation) needs more memory than its task can get. Spark writes intermediate data to local disk and reads it back later. The job survives, but disk I/O makes it slower — so spill is an early warning of memory pressure.

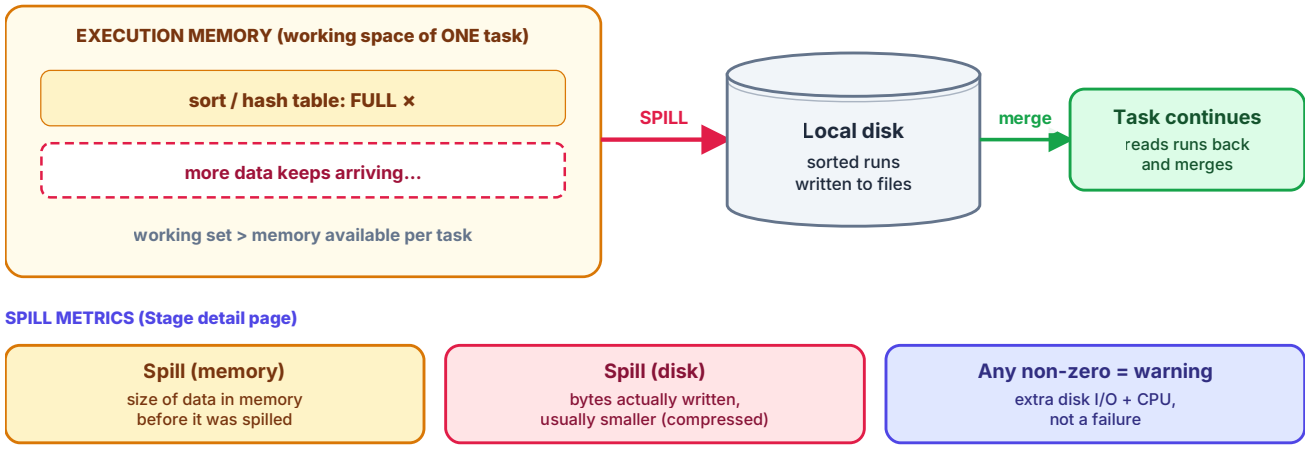


Figure 4.5 — When working data does not fit in execution memory

Cause	Remedy
Too few shuffle partitions (each task gets too much)	Increase `spark.sql.shuffle.partitions` or rely on AQE (2.5)
Skewed key: one task gets a huge share	Skew handling (3.5 – 3.7)
Executors too small for the work	More memory per core (fewer cores per executor, or bigger node)
Too much cached data eating the unified region	Unpersist unused caches; use DISK_ONLY
Wide rows / huge `collect_list` groups	Select fewer columns; avoid giant groups

INTERVIEW ANGLE
How to say it: *"I see spill (memory) and spill (disk) in the stage summary; I reduce it by increasing partitions so each task handles less data, fixing skew, or giving executors more memory per core."* Spill vs OOM: spill slows, OOM kills.

4.6 Executor Sizing: Cores, Memory and Overhead

★★★

IN ONE LINE

Choosing cores and memory per executor so tasks have enough memory without GC or I/O contention.

Executor sizing decides how many tasks run in parallel and how much memory each gets. The three knobs are `spark.executor.cores`, `spark.executor.memory` and `spark.executor.instances` (or dynamic allocation).

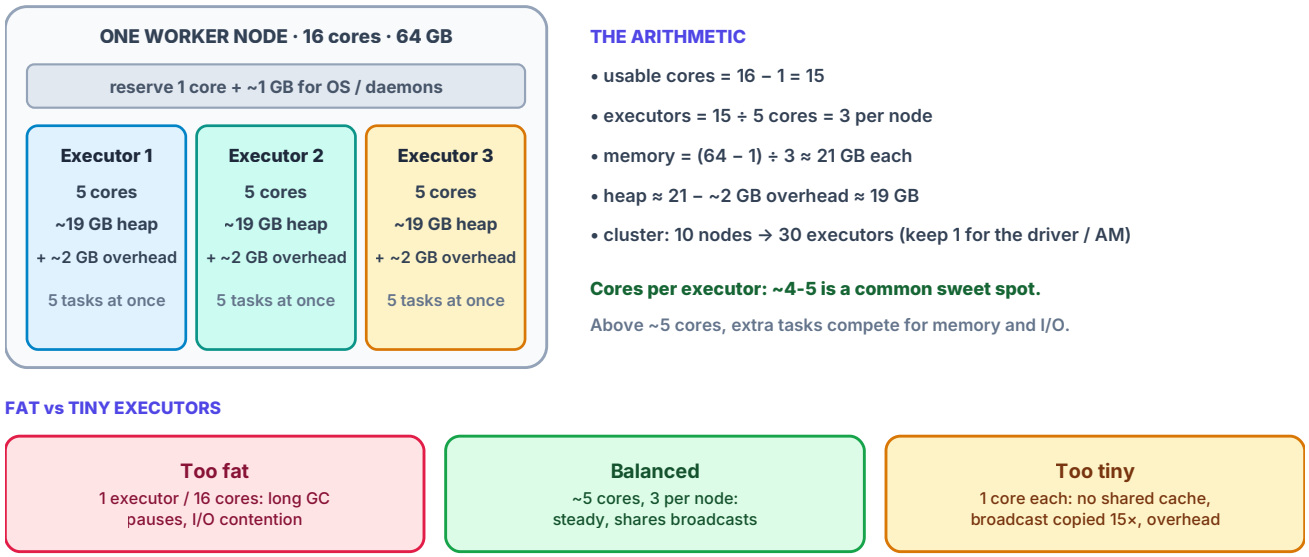


Figure 4.6 — Sizing executors on a 16-core, 64 GB node

SPARK-SUBMIT FLAGS FOR THE EXAMPLE ABOVE

```
spark-submit \
  --num-executors 30 \
  --executor-cores 5 \
  --executor-memory 19g \
  --conf spark.executor.memoryOverhead=2g \
  --driver-memory 8g \
  job.py
```

Knob	Guidance
`executor-cores`	About 4–5. More cores = more tasks share one memory pool.
`executor-memory`	Enough for partition size × cores × working factor; leave room for overhead.
`memoryOverhead`	Raise for PySpark, pandas UDFs, heavy native use (container-kill errors).
Dynamic allocation	`spark.dynamicAllocation.enabled` adds/removes executors with load (shared clusters, variable stages).
Driver	Needs memory if you collect, broadcast or have very many tasks / partitions.

INTERVIEW ANGLE

On Databricks you normally pick node types and worker counts (7.1) and the platform runs one executor per worker using all its cores, so these flags rarely apply. Know the logic for YARN / Kubernetes / EMR interviews.